

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Disciplina: Teste de Software

Análise de Eficácia de Testes com Teste de Mutação

Aluna: **Gabriela Alvarenga Cardoso**

Matrícula: **844469**

Professor: Cleiton Tavares

Repositório: <https://github.com/gabialvarenga/operacoes-mutante>

Outubro de 2026

1. Análise Inicial

O projeto tem 50 funções matemáticas em `src/operacoes.js` e 50 testes em `test/operacoes.test.js`, um por função. Todos passam, e a cobertura medida pelo Jest é alta: 98,64% das linhas e 100% das funções.

```
Windows PowerShell - operacoes-mutante
PS C:\...\operacoes-mutante> npx jest --coverage
-----
File                | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files            | 85.41   | 58.82    | 100     | 98.64   |
operacoes.js        | 85.41   | 58.82    | 100     | 98.64   | 112
-----
Test Suites: 1 passed, 1 total
Tests:       50 passed, 50 total
Snapshots:   0 total
Time:        0.547 s, estimated 1 s
Ran all test suites.
```

Figura 1 – Cobertura inicial: 98,64% das linhas e 100% das funções; só a linha 112 (mediana de array par) não é executada.

A primeira execução do StrykerJS (configurado com o `jest-runner`) gerou 213 mutantes. Desses, 159 foram detectados e 54 não: 42 sobreviveram e 12 nem chegaram a ser executados por algum teste. O **mutation score** foi de **74,65%**.

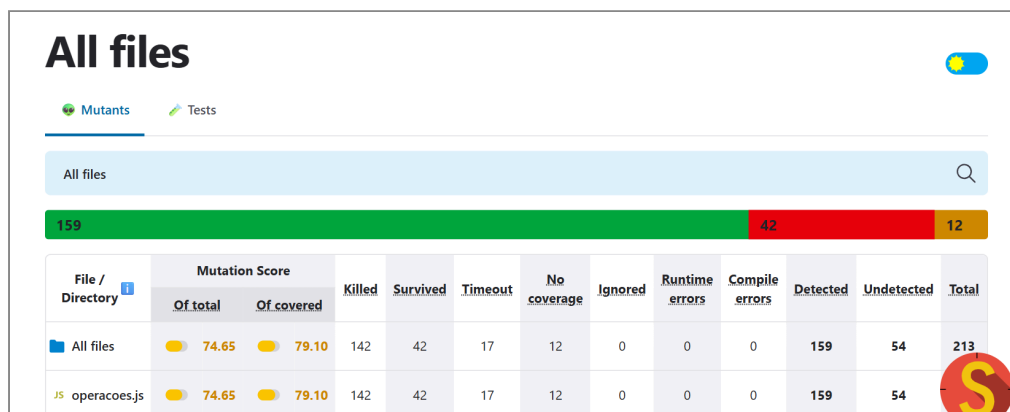


Figura 2 – Relatório inicial do Stryker: mutation score de 74,65%, com 42 mutantes sobreviventes e 12 sem cobertura.

Discrepância: a cobertura só diz que uma linha foi executada, não que o resultado dela foi verificado. Aqui, cada função é testada com um único valor "confortável": `isMaiorQue(10, 5)`, `isPar(100)`, `fatorial(4)`. A linha roda, a cobertura sobe, mas os casos de limite (valores iguais, zero, negativos, array vazio) e o caso `false` das funções booleanas nunca são conferidos. Por isso, quase um quarto dos defeitos artificiais passou despercebido, mesmo com cobertura perto de 100%. A cobertura de *branches* (58,82%) já dava uma pista disso.

2. Análise de Mutantes Críticos

2.1 isMaiorQue: `a > b` trocado por `a >= b`

```
98 }
99
100 // === Bloco 5: Geometria e Lógica de Comparação (41-50) ===
101 function areaCirculo(raio) { return Math.PI * Math.pow(raio, 2); }
102 function areaRetangulo(base, altura) { return base * altura; }
103 function perimetroRetangulo(base, altura) { return 2 * (base + altura); }
104 - function isMaiorQue(a, b) { return a > b; }
105 + function isMaiorQue(a, b) { return a >= b; }
106 function isMenorQue(a, b) { return a < b; }
107 function isEqual(a, b) { return a === b; }
108 function medianaArray(numeros) {
```

Figura 3 – Mutante EqualityOperator sobrevivente na linha 104.

O que a mutação fez: trocou o operador relacional, simulando o erro clássico de "fora por um" na fronteira. **Por que sobreviveu:** o único teste é `isMaiorQue(10, 5)`, e 10 é maior e também maior ou igual a 5. As duas versões só divergem quando `a === b`, caso que a suíte nunca testou. O mesmo aconteceu com `isMenorQue`, e o mutante que troca o corpo por `return true` também sobreviveu, porque nenhum teste esperava `false`.

2.2 medianaArray: ordenação removida

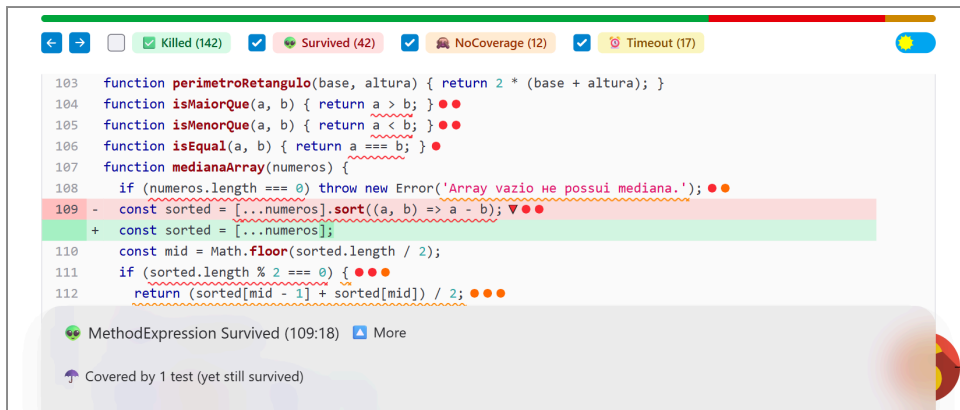


Figura 4 – Mutante MethodExpression sobrevivente na linha 109: o `.sort()` foi apagado.

O que a mutação fez: removeu a chamada a `sort`, então a função passa a pegar o elemento do meio do array na ordem em que ele chegou. **Por que sobreviveu:** o teste usa `[1, 2, 3, 4, 5]`, que já está ordenado; ordenar ou não dá o mesmo resultado. Pelo mesmo motivo sobreviveram as trocas do comparador (`a + b` e `() => undefined`). E, como o array tinha tamanho ímpar, todo o ramo de tamanho par ficou sem cobertura.

2.3 fatorial: `n < 0` trocado por `n <= 0`

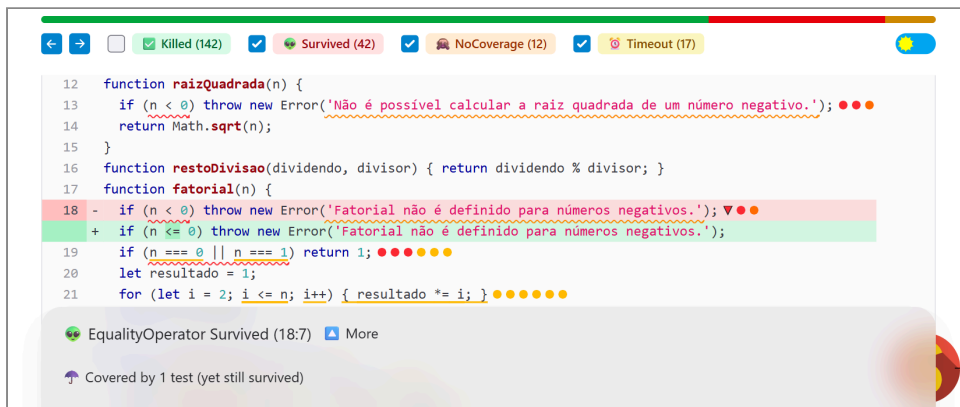


Figura 5 – Mutante EqualityOperator sobrevivente na linha 18.

O que a mutação fez: passou a rejeitar o zero como se fosse negativo, quebrando `fatorial(0) = 1`. **Por que sobreviveu:** o teste só verifica `fatorial(4)`; nem o zero nem um número negativo são testados, então a validação inteira podia ser alterada (ou até trocada por `if (false)`) sem nenhum teste falhar.

3. Solução Implementada

Mantive os 50 testes originais e criei um bloco `describe` com 31 testes novos, cada um mirando um mutante (81 no total). Para os três casos analisados:

```
test('isMaiorQue deve retornar false para valores iguais', () => { expect(isMaiorQue(5, 5)).toBe(false); });
test('isMaiorQue deve retornar false quando a é menor que b', () => { expect(isMaiorQue(5, 10)).toBe(false); });

test('medianaArray deve ordenar o array antes de calcular (tamanho ímpar)', () => { expect(medianaArray([5, 1, 3])).toBe(3); });
test('medianaArray deve tirar a média dos dois elementos centrais (tamanho par)', () => { expect(medianaArray([4, 1, 3, 2])).toBe(2.5); });

test('fatorial de zero deve ser 1 (limite da validação)', () => { expect(fatorial(0)).toBe(1); });
test('fatorial deve lançar erro para número negativo', () => { expect(() => fatorial(-1)).toThrow('Fatorial não é definido para números negativos.');
```

- **Valores de fronteira:** `isMaiorQue(5, 5)` e `fatorial(0)` ficam exatamente no ponto em que `>` e `>=` (ou `<` e `<=`) dão respostas diferentes, então qualquer troca de operador faz o teste falhar.
- **Entradas que dependem da regra:** o array `[5, 1, 3]` só tem mediana 3 se for ordenado (sem ordenação, o meio seria 1). O array `[4, 1, 3, 2]` exercita o ramo par e mata as mutações de `mid - 1`, e `e / 2`.
- **Mensagens de erro exatas:** `toThrow('mensagem')` mata os mutantes que trocam o texto por `""`, que sobreviviam com um `toThrow()` genérico.

Os demais testes seguem as mesmas ideias: o caso false de isPar, isImpar, isDivisivel, isMenorQue e isEqual; isPrimo com 0, 1, 2 e 9; arrays vazios; e a conversão de 100 °C em 212 °F, que expõe a fração 9/5 (com 0 °C qualquer fator dá 32 °F).

Mutantes equivalentes: depois dos novos testes, sobraram 7 mutantes que nenhum teste consegue matar, porque não mudam o comportamento do programa. No fatorial, o if (n === 0 || n === 1) return 1 é redundante, pois o laço já devolve 1 para esses valores (4 mutantes). Em produtoArray, o reduce com valor inicial 1 já devolve 1 para array vazio. Em clamp, trocar < por <= (e > por >=) só muda o caso valor === min, em que devolver min ou valor dá no mesmo. Marquei essas linhas com o comentário // Stryker disable next-line, com a justificativa, sem alterar a lógica do código.

4. Resultados Finais

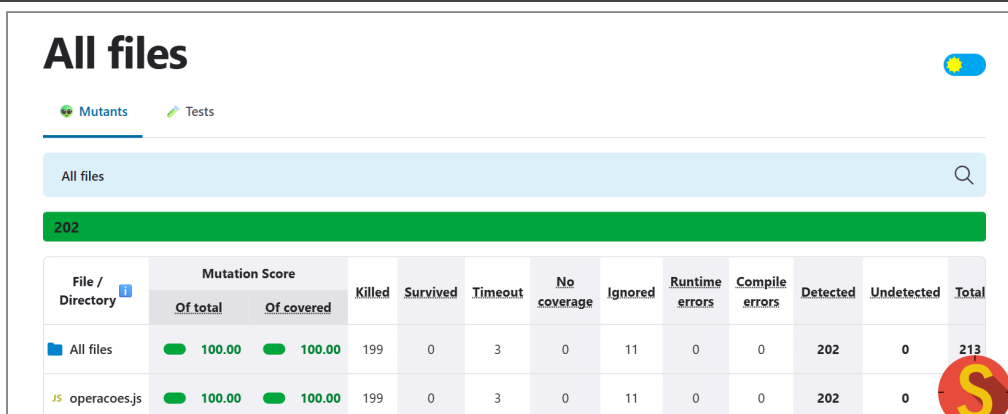


Figura 6 – Relatório final do Stryker: 202 mutantes detectados, nenhum sobrevivente e mutation score de 100%.

Métrica	Inicial	Final
Testes	50	81
Cobertura de linhas / branches	98,64% / 58,82%	100% / 100%
Mutantes sobreviventes / sem cobertura	42 / 12	0 / 0
Mutation score (sem ignorar nenhum mutante)	74,65%	96,71% (só os 7 equivalentes sobrevivem)
Mutation score (com equivalentes ignorados)	—	100%

A meta de mais de 98% foi atingida. Um detalhe de transparência: o comentário Stryker disable vale para a linha inteira, então o relatório mostra 11 mutantes ignorados, e não 7. Os 4 a mais estão na linha do fatorial e já eram mortos pelos novos testes antes de a linha ser marcada. Sem marcar nada, o score real é 96,71%, e toda a diferença para 100% vem dos mutantes equivalentes.

5. Conclusão

O trabalho mostrou na prática que cobertura e eficácia são coisas diferentes. Com quase 99% das linhas cobertas, a suíte original deixava passar um quarto dos defeitos simulados, porque executava o código sem verificar os casos em que ele pode errar. O teste de mutação mede exatamente isso: em vez de perguntar "esta linha rodou?", pergunta "se esta linha estivesse errada, algum teste perceberia?".

Além de dar um número mais honesto, o relatório do Stryker funciona como um guia: cada mutante sobrevivente aponta um teste que falta, quase sempre um valor de fronteira, um caso negativo ou uma entrada que depende da regra de negócio. Os mutantes equivalentes também ensinam algo, porque revelaram código redundante no fatorial. O custo é o tempo de execução, maior que o de uma suíte comum, mas, como ferramenta para avaliar e melhorar a qualidade dos testes, o teste de mutação dá uma confiança que a cobertura sozinha não consegue dar.